

Notes on my Invoice system code

1. Overview

Program basically 3 cheezon ko handle karta hai:

1. Customers
2. Products
3. Invoices

2. Data Structures (Structs)

C++ me struct ka use hota hai related data ko ek jagah group karne ke liye.

Mere program me teen structs use hue hain:

Customer:

```
struct Customer {  
    string id;  
    string name;  
};
```

Har customer ka ek unique id aur name hota hai. Invoice banate waqt customer ko ID se identify kiya jata hai.

Product:

```
struct Product {  
    string id;  
    string name;  
    float price;  
};
```

Har product ka id, name, aur price hota hai. Jab invoice banate hain, user product ka ID enter karta hai aur uska price automatically fetch hota hai.

Item:

```
struct Item {  
    Product product;  
    int quantity;  
};
```

Invoice ke andar ek Item ka matlab hai ek product aur uski quantity. Yahan pe maine nested struct ka concept use kiya hai (Item ke andar Product struct hai).

3. Utility Functions

Maine kuch helper functions banaye jo repetitive kaam simplify kar dete hain:

`getDateTime()`:

- Current system time fetch karta hai.
- Time ko human-readable format me convert karke return karta hai.
- Iska use invoice me "Issued on: DD/MM/YYYY hh:mm:ss" dikhane ke liye hota hai.

`getCustomerById()`:

- Input: customer ID aur customers ka list.
- Customers me loop chalata hai aur match hone par customer return karta hai.
- Agar customer nahi mila toh error handle karta hai.

`getProductById()`:

- Same as customer, lekin products ke liye.
- Product ID se product details fetch karta hai.
- Note: Yeh dono functions linear search karte hain → time complexity $O(n)$.

4. Invoice Creation

Invoice generate karna is program ka main kaam hai. Yeh step-by-step process me hota hai:

1. Customer Selection:

- User se Customer ID li jati hai.
- `getCustomerById()` use hota hai customer verify karne ke liye.

2. Adding Products:

- User repeatedly product ID aur quantity enter karta hai.
- Har product ke liye `getProductById()` call hota hai.
- Ek Item struct create hota hai aur items ke list me add hota hai.

3. Invoice Calculation:

- Har item ka total = price × quantity.
- Grand total = saare items ke total ka sum.
- Currency Rs. me show hoti hai.

4. Invoice Printing:

- Customer ka naam, invoice number aur date-time print hota hai.
- Ek table generate hoti hai: Product Name | Quantity | Unit Price | Total.
- Last me Grand Total show hota hai.

5. File Handling

Maine customers aur products ko files me store kiya hai (customers.txt, products.txt).

- Program start hone pe yeh files read hoti hain.
- Har line se data extract karke Customer aur Product structs banaye jate hain.
- Ye data memory me load hota hai aur invoice banate waqt use hota hai.

Is tarah program dynamic ban jata hai — agar naya customer ya product add karna ho toh code change nahi karna, sirf file update karni hoti hai.

6. Complex Parts Explained

1. Nested Structs:

- Item ke andar Product hai. Matlab ek struct ke andar doosra struct use ho raha hai. Yeh real-world design ko follow karta hai.

2. Search Functions:

- Har baar customer aur product fetch karne ke liye pura list check hota hai. Simple hai but $O(n)$ complexity rakhta hai.

3 . Time Handling:

- `time_t` aur formatting functions ka use hota hai. Yeh thoda low-level programming ka part hai jo directly system clock ke sath interact karta hai.

7. Execution Flow

1. Program start hota hai → customers aur products file se load hote hain.
2. User ek valid Customer ID enter karta hai.
3. User multiple products add karta hai (Product ID + Quantity).
4. Items memory me store hote hain aur total calculate hota hai.
5. Final invoice print hota hai with all details.